

LAB MANUAL

Charotar University of Science and Technology (CHARUSAT)

Chandubhai S Patel Institute of Technology (CSPIT)

Smt. Kundanben Dinsha Patel Department of Information Technology

IT341 Design and Analysis of Algorithm

A.Y. 2019-20

Practical List

Analysis of Program should contain following sub heading(s).

1. **Impact of Input Size on the Performance of Program.** Make Table and Draw graph of Input Size Vs Running Time/Total No of Instructions. Take at least Five Input of Different Size.
2. **Impact of Input Quality on the Performance of Program.** Make Table and Draw graph of Best Case, Worst Case and Average Case Input Quality Vs Running Time/ Total No. of Instructions.
3. **Rate of Growth of Program.** Make Table and Draw Graph of Input Size Vs Instruction(s) Running Maximum No of Time in the Program.
4. **Conclusion** from the above graph or Data Table.

Sr No.	Practical Aim	Hrs												
1	Implement and analyze algorithms given below.	02												
1.1	Fibonacci Series(Iterative and Recursive)													
1.2	Factorial of a given number (Iterative and Recursive)													
2	Analyze algorithms given below.	04												
2.1	Bubble Sort													
2.2	Selection Sort													
2.3	Insertion Sort													
3	Implement and analyze algorithms given below.(Divide and Conquer Strategy)	04												
3.1	Design and implement searching algorithm to find given word from English dictionary using minimum number of comparisons. Also find out time complexity of algorithm.													
3.2	Implement and compare worst case performance of Merge Sort and Quick Sort.													
4	Implement and analyze below given problem.(Greedy Approach)	06												
4.1	A cashier at any mall needs to give change of an amount to customers many times in a day. Cashier has multiple number of coins available with different denominations which is described by a set C. Implement the program for a cashier to find the minimum number of coins required to find a change of a particular amount A. Output should be the total number of coins required of given denomination. Check the program for following test cases: <table border="1"> <thead> <tr> <th>Test Case</th><th>Coin denominations C</th><th>Amount A</th></tr> </thead> <tbody> <tr> <td>1</td><td>₹1, ₹2, ₹3</td><td>₹ 5</td></tr> <tr> <td>2</td><td>₹18, ₹17, ₹5, ₹1</td><td>₹ 22</td></tr> <tr> <td>3</td><td>₹100, ₹25, ₹10, ₹5, ₹1</td><td>₹ 289</td></tr> </tbody> </table>	Test Case	Coin denominations C	Amount A	1	₹1, ₹2, ₹3	₹ 5	2	₹18, ₹17, ₹5, ₹1	₹ 22	3	₹100, ₹25, ₹10, ₹5, ₹1	₹ 289	
Test Case	Coin denominations C	Amount A												
1	₹1, ₹2, ₹3	₹ 5												
2	₹18, ₹17, ₹5, ₹1	₹ 22												
3	₹100, ₹25, ₹10, ₹5, ₹1	₹ 289												
4.2	Is the output of Test case 2 is optimal? Write your observation. A Burglar has just broken into the Fort! He sees himself in a room with n piles of gold dust. Because the each pile has a different purity, each pile also has a different value (v[i]) and a different weight (w[i]). A Burglar has a bag that can only hold W kilograms.													

Given $n=5$, $v=\{4,2,2,1,10\}$, $c=\{12,1,2,1,4\}$ and $W=15$, calculate which piles Burglar should completely put into his bag and which he should put only fraction into his bag. Design and implement an algorithm to get maximum piles of gold using given bag with W capacity, Burglar is also allowed to take fractional of pile.

- 4.3 There are eleven Professors in a Department. Each professor wants to deliver lecture in same day. Each professor has some time limits for lecture. Professor earns credit if and only if lecture is arranged on or before its time limit.

Professor	1	2	3	4	5	6	7	8	9	10	11
credit	78	90	50	60	75	10	80	55	88	74	59
Lecture Limit	5	4	5	3	2	1	4	6	4	5	6

Design and implement greedy approach to schedule maximum number of lectures in the department without and with credit.

5 Implement and analyze given problems (Dynamic Programming)

06

- 5.1 Given two integer arrays $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item, or don't pick it (0-1 property).
- 5.2 Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i=1, 2, \dots, n$ matrix A_i with dimensions. Implement the program to fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications. Also calculate the number of scalar multiplications for all possible combinations of matrices.

Test Case	n	Matrices with dimensions
1	3	$A_1: 3 \times 5, A_2: 5 \times 6, A_3: 6 \times 4$
2	6	$A_1: 30 \times 35, A_2: 35 \times 15, A_3: 15 \times 5, A_4: 5 \times 10, A_5: 10 \times 20, A_6: 20 \times 25$

- 5.3 Implement a program which has $BNMCOEF()$ function that takes two parameters n and k and returns the value of Binomial Coefficient $C(n, k)$. Compare the dynamic programming implementation with recursive implementation of $BNMCOEF()$. (In output, entire table should be displayed.)

Test Case	n	k
1	5	2
2	11	6
3	12	5

- 5.4 Implement a program to print the longest common subsequence for the following

strings:

Test Case	String1	String2
1	ABCDAB	BDCABA
2	EXPONENTIAL	POLYNOMIAL
3	LOGARITHM	ALGORITHM

6 String Matching Problem.

02

- 6.1 Suppose you are given a source string $S[0..n-1]$ of length n , consisting of symbols a and b . Suppose further that you are given a pattern string $P[0..m-1]$ of length $m < n$, consisting of symbols a , b , and $*$, representing a pattern to be found in string S . The symbol $*$ is a “wild card” symbol, which matches a single symbol, either a or b . The other symbols must match exactly. The problem is to output a sorted list M of valid “match positions”, which are positions j in S such that pattern P matches the substring $S[j..j+|P|-1]$. For example, if $S = ababbab$ and $P = ab*$, then the output M should be $[0, 2]$.

Implement Naive and Rabin karp algorithm to solve the problem.

7 Implement and analyze the problem.

02

- 7.1 Eight Queen Problem

8 Design and analyze below problems (Graph)

04

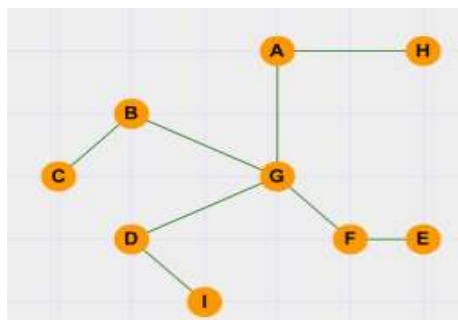
- 8.1 Given an undirected graph and a number m , determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with same color. Here coloring of a graph means assignment of colors to all vertices. Solve this using Backtracking.

- 8.2 Write a program to detect cycles in directed graph.

- 8.3 Let us consider the situation of a war. In your country, there is a network of telephone lines between 9 cities (A, B, C...H, I) i.e. the 9 cities are connected by telephone line, which means that a message from one city to any other city can be transmitted through the line. Like we can transfer message from city A to city B even though they are not "directly" connected by a line. So what's the catch, everything seems fine, right?

You are the "army-general" of your country and you've to take a decision, you have to find the city which, if damaged would incur the greatest network blockage (Considering that damaging the city damages all the connected telephone lines in it).

Implement this situation and find out Which city would you try to protect the most and why?



1. Analyze the problem using No of Instructions
 1. Implement a problem in any language.
 2. Count the no of instruction of the program for at least different 5 input and make a table of Input Size Vs No of Instructions.
 3. Draw the graph of Input size Vs No of Instructions.
 4. Repeat step 2 and 3 for Best Case, Average Case and Worst Case.
 5. Compare the practical complexity with theoretical complexity.
 6. Conclude from above graph or data table.

Analysis Table:-

<u>Input</u>	<u>Best Case</u>			<u>Average Case</u>			<u>Worst Case</u>		
	Instruction	Compare	Exchange	Instruction	Compare	Exchange	Instruction	Compare	Exchange
5									
10									
15									
20									
25									

COMPLEXITY:-

	<u>INSTRUCTION</u>			<u>COMPARE</u>			<u>EXCHANGE</u>		
	BEST CASE	AVERAGE CASE	WORST CASE	BEST CASE	AVERAGE CASE	WORST CASE	BEST CASE	AVERAGE CASE	WORST CASE
Practical Complexity									
Theoretical Complexity									

Practical: 1

Aim: Implement and analyze algorithms given below.

1.1 Fibonacci Series (Iterative and Recursive)

1.2 Factorial of a given number (Iterative and Recursive)

Knowledge Required: Concepts of Iteration and Recursion

Algorithm:

1.1 Fibonacci series

Fibonacci series is defined as a sequence of numbers in which the first two numbers are 1 and 1, or 0 and 1, depending on the selected beginning point of the sequence, and each subsequent number is the sum of the previous two. So, in this series, the n th term is the sum of $(n-1)$ th term and $(n-2)$ th term.

Mathematically, the n th term of the Fibonacci series can be represented as:

$$t_n = t_{n-1} + t_{n-2}$$

The Fibonacci numbers upto certain term can be represented as:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.....

Iterative:

START

Procedure Fibonacci(n)

 declare $f0$, $f1$, fib , $loop$

 set $f0$ to 0

 set $f1$ to 1

 display $f0$, $f1$

 for $loop \leftarrow 1$ to n

$fib \leftarrow f0 + f1$

$f0 \leftarrow f1$

$f1 \leftarrow fib$

 display fib

 end for

END

Recursive:

START

Procedure Fibonacci(n)

declare f0, f1, fib, loop

set f0 to 0

set f1 to 1

display f0, f1

for loop $\leftarrow$ 1 to nfib $\leftarrow$ f0 + f1f0 $\leftarrow$ f1f1 $\leftarrow$ fib

display fib

end for

END

1.2 Factorial number

Factorial of a non-negative integer, is multiplication of all integers smaller than or equal to n. For example factorial of 6 is $6*5*4*3*2*1$ which is 720.

Iterative_Factorial(n)

1: Read number n.

2. Initialize i and fact to 1.

3. Repeat step 4 and step 5 while i is not equal to n.

4. fact $\leftarrow$ fact * i5. i $\leftarrow$ i +1

6. Return fact

Recursive_Factorial(n)

Begin

if n = 1 then

Return 1;

else

Return n*Call Fact(n-1);

endif

End

Program:**Analysis:**

(Analysis Table and Graph)

Conclusion:

Questions:

1)

Practical: 2

Aim: Analyze algorithms given below.

2.1 Bubble Sort

2.2 Selection Sort

2.3 Insertion Sort

Knowledge Required: Concepts of Sorting

Algorithm:

•Bubble Sort:

This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where n is the number of items.

```
procedure bubbleSort( list : array of items )
```

```
    loop = list.count;
```

```
    for i = 0 to loop-1 do:
```

```
        swapped = false
```

```
        for j = 0 to loop-1 do:
```

```
            /* compare the adjacent elements */
```

```
            if list[j] > list[j+1] then
```

```
                /* swap them */
```

```
                swap( list[j], list[j+1] )
```

```
                swapped = true
```

```
            end if
```

```
        end for
```

```
        if(not swapped) then
```

```
            break
```

```
        end if
```

```
    end for
```

```
end procedure return list
```

•Selection Sort:

This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where n is the number of items.

procedure selection sort

list : array of items

n : size of list

for i = 1 to n - 1

/* set current element as minimum*/

min = i

/* check the element to be minimum */

for j = i+1 to n

if list[j] < list[min] then

min = j;

end if

end for

/* swap the minimum element with the current element*/

if indexMin != i then

swap list[min] and list[i]

end if

end for

end procedure

•Insertion Sort:

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element which is to be 'insert'ed in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, insertion sort.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where n is the number of items.

INSERTION-SORT(A)

for $j \leftarrow 2$ to n

do $key \leftarrow A[j]$

 Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$

$i \leftarrow j - 1$

 while $i > 0$ and $A[i] > key$

 do $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow key$

Program:

Analysis:

Conclusion:

Questions:

Practical: 3

Aim: Implement and analyze algorithms given below. (Divide and Conquer Strategy)

3.1 Design and implement searching algorithm to find given word from English dictionary using minimum number of comparisons. Also find out time complexity of algorithm.

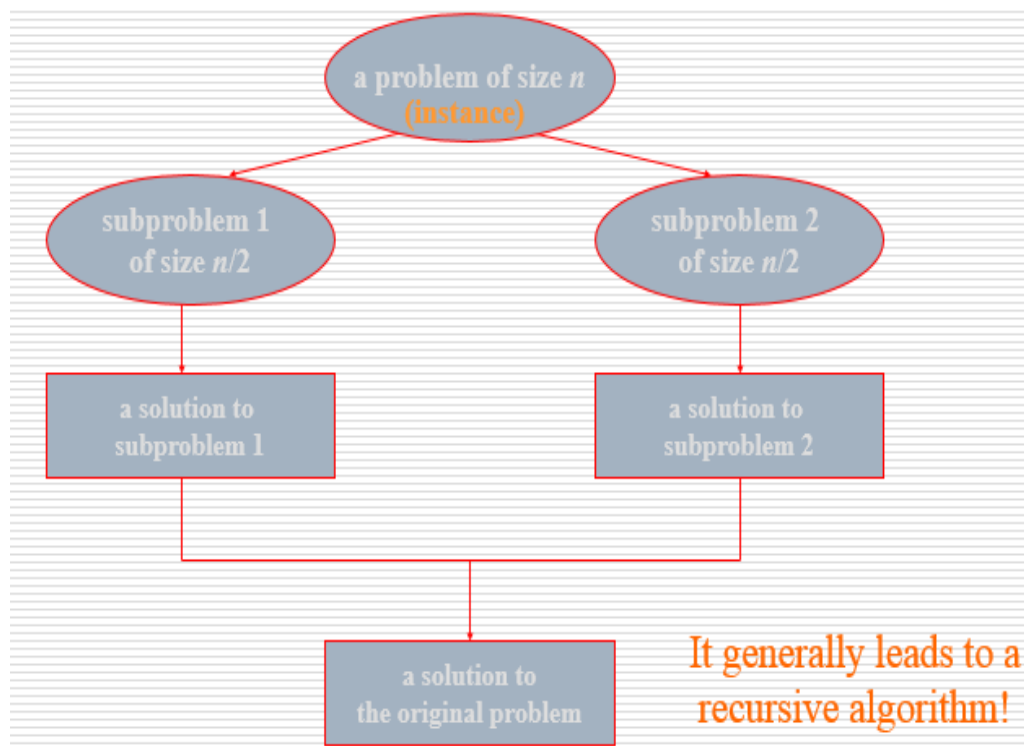
3.2 Implement and compare worst case performance of Merge Sort and Quick Sort.

Knowledge Required: Concepts of Divide and Conquer Strategy

Algorithm:

Structure of Divide and Conquer

- ☐ Divide
 - Break a problem into sub problem(s)
- ☐ Conquer
 - Solve all sub problem(s)
- ☐ Combine
 - Solve the problem using the results of the sub problem(s)



Binary Search:

Problem: Determine whether x is in the sorted array S of size n .

Inputs: positive integer n , sorted (nondecreasing order) array of keys S indexed from 1 to n , a key x .

Outputs: *location*, the location of x in S (0 if x is not in S).

```

index location (index low, index high)
{
    index mid;
    if (low > high)
        return 0;
    else {
        mid = [(low + high)/2];
        if (x == S[mid])
            return mid;
        else if (x < S[mid])
            return location(low, mid - 1);
        else
            return location(mid + 1, high);
    }
}

```

Merge Sort

☐ Divide:

If S has at least two elements, remove all the elements from S and put them into two sequences, S_1 and S_2 , each containing about half of the elements of S .

☐ Conquer:

Sort sequences S_1 and S_2 using Merge Sort.

☐ Combine:

Put back the elements into S by merging the sorted sequences S_1 and S_2 into one sorted sequence

```

Merge-Sort(A, p, r)
    if p < r then
        q ← (p+r) / 2
        Merge-Sort(A, p, q)
        Merge-Sort(A, q+1, r)
        Merge(A, p, q, r)

```

Merge(A, p, q, r)

Take the smallest of the two topmost elements of sequences $A[p..q]$ and $A[q+1..r]$ and put into the resulting sequence. Repeat this, until both sequences are empty. Copy the resulting sequence into $A[p..r]$.

Quick Sort

Given an array of n elements (e.g., integers):

- ☐ If array only contains one element, return
- ☐ Else
 - pick one element to use as *pivot*.
 - Partition elements into two sub-arrays:
 - ☐ Elements less than or equal to pivot
 - ☐ Elements greater than pivot
 - Quicksort two sub-arrays
 - Return results

Steps:

1. While `data[too_big_index] <= data[pivot]`
 `++too_big_index`
2. While `data[too_small_index] > data[pivot]`
 `--too_small_index`
3. If `too_big_index < too_small_index`
 swap `data[too_big_index]` and `data[too_small_index]`

Questions:

Practical: 4

Aim: Implement and analyze below given problem. (Greedy Approach)
4.1, 4.2, 4.3

Knowledge Required: Concepts of Greedy Approach

Algorithm:

A Generic Greedy Algorithm: Greedy(C)

- (1) Initialize C to be the set of candidate solutions
- (2) Initialize a set S = the empty set $\emptyset$
(the set is to be the optimal solution we are constructing).
- (3) While $C \neq \emptyset$ and S is (still) not a solution do
 - (3.1) select x from set C using a greedy strategy
 - (3.2) x removed from C
 - (3.3) if $\{x\} \cup S$ is a feasible solution, then
 $S = S \cup \{x\}$ (i.e., add x to set S)
- (4) if S is a solution then
 return S
- (5) else return failure

4.1 MAKE-CHANGE (n)

Make change for N units using the least possible number of coins.

```

C ← {100, 25, 10, 5, 1} // constant.
Sol ← {}; // set that will hold the solution set.
Sum ← 0 sum of item in solution set
WHILE sum not = n
  x = largest item in set C such that
    sum + x ≤ n
  IF no such item THEN
    RETURN "No Solution"
  S ← S ∪ {value of x}
  sum ← sum + x
RETURN S
  
```

Program:

Analysis:

4.2 Knapsack Problem

Given n objects each have a weight w_i and a value v_i , and given a knapsack of total capacity W .

Input: an integer n , positive values w_i and v_i , for $1 \leq i \leq n$, and another positive value W .

Output: n values x_i such that $0 \leq x_i \leq 1$ and

$$\sum_{i=1}^n x_i w_i \leq W \text{ and } \sum_{i=1}^n x_i v_i \text{ is maximized.}$$

Algorithm

- (1) Sort the n objects from large to small based on the ratios v_i/w_i . We assume the arrays $w[1..n]$ and $v[1..n]$ store the respective weights and values after sorting.
- (2) initialize array $x[1..n]$ to zeros.
- (3) $\text{weight} = 0$; $i = 1$
- (4) while ($i \leq n$ and $\text{weight} < W$) do
 - (4.1) if $\text{weight} + w[i] \leq W$ then $x[i] = 1$
 - (4.2) else $x[i] = (W - \text{weight}) / w[i]$
 - (4.3) $\text{weight} = \text{weight} + x[i] * w[i]$
 - (4.4) $i++$

Program:

Analysis:

4.3 Job sequencing with deadlines

- (1) Sort p_i into non increasing order. After sorting $p_1 \geq p_2 \geq p_3 \geq \dots \geq p_i$.
- (2) Add the next job i to the solution set if i can be completed by its deadline. Assign i to time slot $[r-1, r]$, where r is the largest integer such that $1 \leq r \leq d_i$ and $[r-1, r]$ is free.
- (3) Stop if all jobs are examined. Otherwise, go to step 2.

Program:

Analysis:

Conclusion:

Questions:

Practical: 5

Aim: Implement and analyze given problems (Dynamic Programming)

5.1, 5.2, 5.3, 5.4

Knowledge Required: Concepts of Dynamic Programming

5.1 Given two integer arrays $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item, or don't pick it (0-1 property).

- i. In 0/1 Knapsack problem, items can be entirely accepted or rejected.
- ii. Given a knapsack with maximum capacity W , and a set S consisting of n items.
- iii. Each item i has some weight w_i and benefit value b_i (all w_i and W are integer values).
- iv. The problem is how to pack the knapsack to achieve maximum total value of packed items.
- v. For solving the knapsack problem we can generate the sequence of decisions in order to obtain the optimum selection.
- vi. Let X_n be the optimum sequence and there are two instances $\{X_n\}$ and $\{X_{n-1}, X_{n-2} \dots X_1\}$.
- vii. So from $\{X_{n-1}, X_{n-2} \dots X_1\}$ we will choose the optimum sequence with respect to X_n .
- viii. The remaining set should fulfill the condition of filling Knapsack of capacity W with maximum profit.
- ix. Thus, 0/1 Knapsack problem is solved using the principle of optimality

To solve this problem using dynamic programming method we will perform following steps:

Steps:

1. Let, $f_i(y_j)$ be the value of optimal solution.
2. Using
formula: $f_i(y_j) = \max\{f_{i-1}(y), f_{i-1}(y - w_i) + p_i\}$ to solve problem.
3. Then S_i is a pair (p, w) where $p = f_i(y_j)$ and $w = y_j - w_i$
4. Initially $S_0 = (0, 0)$
5. Then $S_{i+1} = (P, W) \mid (P - p_i, W - w_i)$
6. S_{i+1} can be computed by merging S_i and S_{i+1}
7. This is used for obtaining optimal solution.

Algorithm:

```

FOR w = 0 TO W
  DO c[0, w] = 0
FOR i=1 to n
  DO c[i, 0] = 0
  FOR w=1 TO W
    DO IF  $w_i \leq w$ 
      THEN IF  $v_i + c[i-1, w-w_i]$ 
        THEN  $c[i, w] = v_i + c[i-1, w-w_i]$ 
        ELSE  $c[i, w] = c[i-1, w]$ 
      ELSE
         $c[i, w] = c[i-1, w]$ 

```

PROGRAM:**ANALYSIS:**

This dynamic-0-1-kanpsack algorithm takes $\theta(nw)$ times, broken up as follows: $\theta(nw)$ times to fill the c -table, which has $(n+1).(w+1)$ entries, each requiring $\theta(1)$ time to compute. $O(n)$ time to trace the solution, because the tracing process starts in row n of the table and moves up 1 row at each step.

5.2 Given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i=1, 2, \dots, n$ matrix A_i with dimensions. Implement the program to fully parenthesize the product $A_1, A_2, \dots, A_n$ in a way that minimizes the number of scalar multiplications.

Also calculate the number of scalar multiplications for all possible combinations of matrices.

Test Case	n	Matrices with dimensions
1	3	A1: 3*5, A2: 5*6, A3: 6*4
2	6	A1: 30*35, A2: 35*15, A3: 15*5, A4: 5*10, A5: 10*20, A6: 20*25

Knowledge Required:

Matrix chain multiplication (or Matrix Chain Ordering Problem, MCOP) is an optimization problem that can be solved using dynamic programming. Given a sequence of matrices, the goal is to find the most efficient way to multiply these matrices. The problem is not actually to *perform* the multiplications, but merely to decide the sequence of the matrix multiplications involved.

There are many options because matrix multiplication is associative. In other words, no matter how the product is parenthesized, the result obtained will remain the same. For example, for four matrices A, B, C , and D , we would have:

$$((AB)C)D = (A(BC))D = (AB)(CD) = A((BC)D) = A(B(CD)).$$

However, the order in which the product is parenthesized affects the number of simple arithmetic operations needed to compute the product, which is the computational complexity. For example, if A is a 10×30 matrix, B is a 30×5 matrix, and C is a 5×60 matrix, then computing $(AB)C$ needs $(10 \times 30 \times 5) + (10 \times 5 \times 60) = 1500 + 3000 = 4500$ operations, while computing $A(BC)$ needs $(30 \times 5 \times 60) + (10 \times 30 \times 60) = 9000 + 18000 = 27000$ operations.

ANALYSIS:

Clearly, the space complexity of this procedure is $O(n^2)$. Since the tables m and s require $O(n^2)$ space. As far as the time complexity is concerned, a simple inspection of the for-loop(s) structures gives us a running time of the procedure. Since, the three for-loops are nested three deep, and each one of them iterates at most n times (that is to say indices L , i , and j takes on at most $n - 1$ values). Therefore, The running time of this procedure is $O(n^3)$.

5.3 Implement a program which has `BNMCOEF()` function that takes two parameters n and k and returns the value of Binomial Coefficient $C(n, k)$. Compare the dynamic programming implementation with recursive implementation of `BNMCOEF()`. (In output, entire table should be displayed.)

Test Case	n	k
1	5	2
2	11	6

Knowledge Required:

Binomial coefficients $\binom{n}{k}$ are the number of ways to select a set of k elements from n different elements without taking into account the order of arrangement of these elements (i.e., the number of unordered sets).

Binomial coefficients are also the coefficients in the expansion of $(a+b)^n$ (so-called binomial theorem):

$$(a+b)^n = \binom{n}{0}a^n + \binom{n}{1}a^{n-1}b + \binom{n}{2}a^{n-2}b^2 + \dots + \binom{n}{k}a^{n-k}b^k + \dots + \binom{n}{n}b^n$$

Algorithm:

Binomial(n, k)

```

for  $i \leftarrow 0$  to  $n$  do // fill out the table row wise
    for  $j = 0$  to  $\min(i, k)$  do
        if  $j=0$  or  $j=i$  then  $C[i, j] \leftarrow 1$  // IC
        else  $C[i, j] \leftarrow C[i-1, j-1] + C[i-1, j]$  // recursive relation
    return  $C[n, k]$ 

```

Program:**Analysis:**

$$\begin{aligned}
A(n, k) &= \text{sum for upper triangle} + \text{sum for the lower rectangle} \\
&= \sum_{i=1}^k \sum_{j=1}^{i-1} 1 + \sum_{i=1}^n \sum_{j=1}^k 1 \\
&= \sum_{i=1}^k (i-1) + \sum_{i=1}^n k \\
&= (k-1)k/2 + k(n-k) \in \Theta(nk)
\end{aligned}$$

Time Complexity: $O(n*k)$

5.4 Implement a program to print the longest common subsequence for the following strings:

Test Case	String1	String2
1	ABCDAB	BDCABA
2	EXPONENTIAL	POLYNOMIAL
3	LOGARITHM	ALGORITHM

Knowledge Required:

Following is detailed algorithm to print the LCS. It uses the same 2D table $L[][]$.

- 1) Construct $L[m+1][n+1]$ using the steps discussed in previous post.
- 2) The value $L[m][n]$ contains length of LCS. Create a character array `lcs[]` of length equal to the length of lcs plus 1 (one extra to store `\0`).
- 2) Traverse the 2D array starting from $L[m][n]$. Do following for every cell $L[i][j]$
 -a) If characters (in X and Y) corresponding to $L[i][j]$ are same (Or $X[i-1] == Y[j-1]$), then include this character as part of LCS.
 -b) Else compare values of $L[i-1][j]$ and $L[i][j-1]$ and go in direction of greater value.

Algorithm: LCS-Length-Table-Formulation (X, Y)

```

m := length(X)
n := length(Y)
for i = 1 to m do
    C[i, 0] := 0
for j = 1 to n do
    C[0, j] := 0
for i = 1 to m do
    for j = 1 to n do
        if  $x_i = y_j$ 
            C[i, j] := C[i - 1, j - 1] + 1
            B[i, j] := 'D'
        else

```

```

    if  $C[i-1, j] \geq C[i, j-1]$ 
         $C[i, j] := C[i-1, j] + 1$ 
         $B[i, j] := 'U'$ 
    else
         $C[i, j] := C[i, j-1]$ 
         $B[i, j] := 'L'$ 
return C and B

```

Algorithm: Print-LCS (B, X, i, j)

```

if  $i = 0$  and  $j = 0$ 
    return
if  $B[i, j] = 'D'$ 
    Print-LCS(B, X,  $i-1, j-1$ )
    Print( $x_i$ )
else if  $B[i, j] = 'U'$ 
    Print-LCS(B, X,  $i-1, j$ )
else
    Print-LCS(B, X,  $i, j-1$ )

```

Program:**Analysis:**

To populate the table, the outer for loop iterates m times and the inner for loop iterates n times. Hence, the complexity of the algorithm is $O(m, n)$, where m and n are the length of two string

Conclusion:**Questions:**

Practical: 6

Aim: String Matching Problem

Suppose you are given a source string $S[0 \dots n - 1]$ of length n , consisting of symbols a and b . Suppose further that you are given a pattern string $P[0 \dots m - 1]$ of length $m < n$, consisting of symbols a , b , and $*$, representing a pattern to be found in string S . The symbol $*$ is a “wild card” symbol, which matches a single symbol, either a or b . The other symbols must match exactly. The problem is to output a sorted list M of valid “match positions”, which are positions j in S such that pattern P matches the substring $S[j..j + |P| - 1]$. For example, if $S = ababbab$ and $P = ab*$, then the output M should be $[0, 2]$.

Implement Naive and Rabin karp algorithm to solve the problem

Knowledge Required: Concepts of String Matching Problem

Algorithm:

RABIN-KARP-MATCHER (T,P,d,q)

1. $n = T.length$
2. $m = P.length$
3. $h = d^{m-1} \bmod q$
4. $p = 0$
5. $t_0 = 0$
6. for $i = 1$ to m
7. $p = (dp + P[i]) \bmod q$
8. $t_0 = (d t_0 + T[i]) \bmod q$
9. For $s = 0$ to $n - m$
10. if $p == t_s$
11. if $P[1 \dots m] == T[s+1 \dots s+m]$
12. printf “Pattern occurs with shift” s
13. if $s < n - m$
14. $t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$

Program:

Analysis:

Conclusion:

Questions:

Practical: 7

Aim: Implement and analyze the problem.

7.1 Eight Queen Problem

Knowledge Required: Concepts of Backtracking

Algorithm:

NQueens(1,n)

```
void NQueens(int k, int n)
// Using backtracking, this procedure prints all
// possible placements of n queens on an n x n
// chessboard so that they are non attacking.
{
    for (int i=1; i<=n; i++) {
        if (Place(k, i)) {
            x[k] = i;
            if (k==n) { for (int j=1; j<=n; j++)
                        cout << x[j] << ' '; cout << endl;}
            else NQueens(k+1, n);
        }
    }
}

bool Place(int k, int i)
// Returns true if a queen can be placed in kth row and
// ith column. Otherwise it returns false. x[] is a
// global array whose first (k-1) values have been set.
// abs(r) returns the absolute value of r.
{
    Two in the same column
    for (int j=1; j < k; j++)
        if ((x[j] == i) || (abs(x[j]-i) == abs(j-k)))
            return(false);
    return(true);
}
```

Program:

Analysis:

Conclusion:

Questions:

Practical: 8

**Aim: Design and analyze below problems (Graph).
8.1, 8.2, 8.3**

Knowledge Required: Concepts of Graph

8.1 Given an undirected graph and a number m , determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with same color. Here coloring of a graph means assignment of colors to all vertices. Solve this using Backtracking.

Knowledge Required: “Graph theory” and “Graph coloring”

In graph theory, graph coloring is a special case of graph labeling; it is an assignment of labels traditionally called "colors" to elements of a graph subject to certain constraints. In its simplest form, it is a way of coloring the vertices of a graph such that no two adjacent vertices are of the same color; this is called a vertex coloring. Similarly, an edge coloring assigns a color to each edge so that no two adjacent edges are of the same color, and a face coloring of a planar graph assigns a color to each face or region so that no two faces that share a boundary have the same color.

Vertex coloring is the starting point of graph coloring. Other coloring problems can be transformed into a vertex version. For example, an edge coloring of a graph is just a vertex coloring of its line graph, and a face coloring of a plane graph is just a vertex coloring of its dual. However, non-vertex coloring problems are often stated and studied as is. That is partly for perspective, and partly because some problems are best studied in non-vertex form, as for instance is edge coloring.

Algorithm: Generate all possible configurations of colors and print a configuration that satisfies the given constraints.

```
while there are untried configurations
{
    generate the next configuration
    if no adjacent vertices are colored with same color
    {
        print this configuration;
    }
}
```

Program:

Analysis:

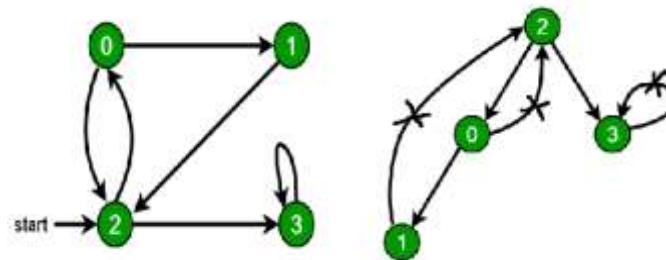
8.2 Write a program to detect cycles in directed graph.

Knowledge Required: Graph Data Structure And Algorithms

Depth First Traversal can be used to detect a cycle in a Graph. DFS for a connected graph produces a tree. There is a cycle in a graph only if there is a back edge present in the graph. A back edge is an edge that is from a node to itself (self-loop) or one of its ancestor in the tree produced by DFS. In the following graph, there are 3 back edges, marked with a cross sign. We can observe that these 3 back edges indicate 3 cycles present in the graph.

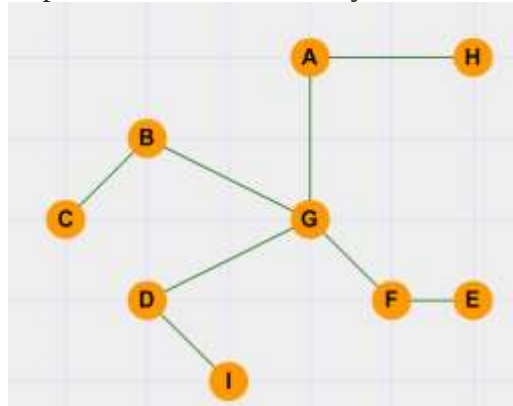
For a disconnected graph, we get the DFS forest as output. To detect cycle, we can check for a cycle in individual trees by checking back edges.

To detect a back edge, we can keep track of vertices currently in recursion stack of function for DFS traversal. If we reach a vertex that is already in the recursion stack, then there is a cycle in the tree. The edge that connects current vertex to the vertex in the recursion stack is a back edge. We have used recStack[] array to keep track of vertices in the recursion stack.

**Algorithm:****Program:****Conclusion:****Questions:**

8.3 Let us consider the situation of a war. In your country, there is a network of telephone lines between 9 cities (A, B, C...H, I) i.e. the 9 cities are connected by telephone line, which means that a message from one city to any other city can be transmitted through the line. Like we can transfer message from city A to city B even though they are not directly connected by a line. So what's the catch, everything seems fine, right? You are the army-general of your country and you have to take a decision, you have to find the city which, if damaged would incur the greatest network blockage (Considering that damaging the city

damages all the connected telephone lines in it). Implement this situation and find out which city would you try to protect the most and why?



Knowledge Required: Depth-first-search(DFS)

Algorithm:

Program:

Output:

Conclusion:

Questions: